

ПРАКТИЧНА РОБОТА № 5

ШАБЛОНИ ПОВЕДІНКИ. ШАБЛОНИ VISITOR, MEMENTO, STATE, COMMAND, INTERPRETER.

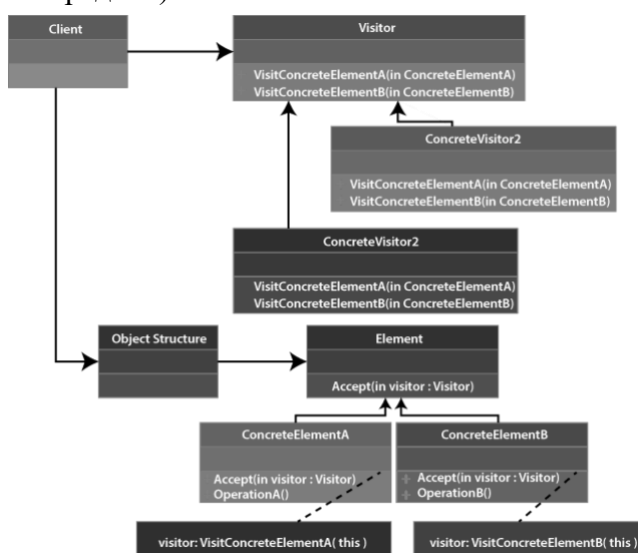
Мета: Вивчення шаблонів поведінки. Отримання базових навичок з застосування шаблонів **Visitor**, **Memento**, **State**, **Command**, **Interpreter**.

Короткі теоретичні відомості

Шаблон Visitor

Проблема. Над кожним об'єктом деякої структури виконується операція. Визначити нову операцію, не змінюючи класи об'єктів.

Рішення. Клієнт, який використовує даний шаблон, повинен створити об'єкт класу "ConcreteVisitor", а потім відвідати кожен елемент структури. "Visitor" оголошує операцію "VisitConcreteElement" для кожного класу "ConcreteElement" (ім'я та сигнатура даної операції ідентифікують клас, елемент якого відвідує "Visitor" – тобто, відвідувач може звертатися до елементу безпосередньо).



Мал. 1. Структура шаблону Visitor

Учасники шаблону:

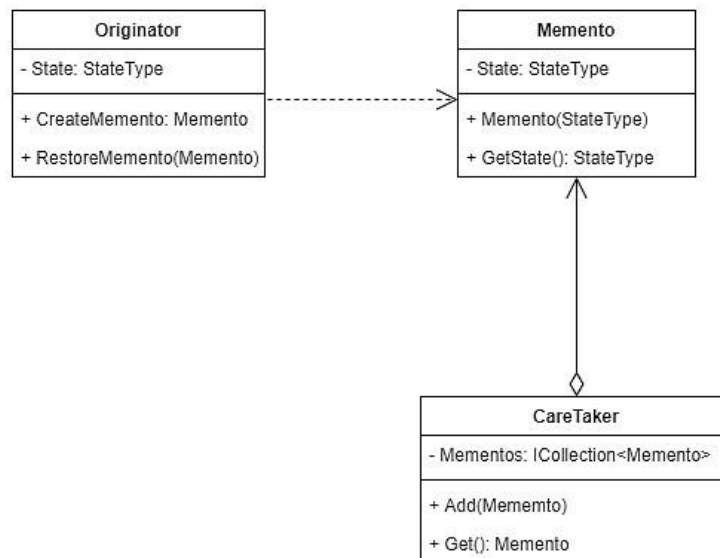
1. **Visitor (Відвідувач)** – це інтерфейс або абстрактний клас, який визначає набір методів відвідувача, кожен з яких відповідає певній операції.
2. **ConcreteVisitor (Конкретний відвідувач)**. Кожен конкретний відвідувач реалізує операції для конкретних типів об'єктів.
3. **Element (Елемент)** – це інтерфейс або абстрактний клас, який визначає метод асепт(), приймаючи відвідувача як параметр. Цей метод використовується для передачі самого себе відвідувачу.
4. **ConcreteElement (Конкретний елемент)**. Реалізація інтерфейсу або підклас абстрактного класу Element. Кожен конкретний елемент реалізує метод асепт() та може мати власний стан.

ObjectStructure (Об'єктна структура) – це клас або структура, яка зберігає колекцію об'єктів елементів і надає інтерфейс для відвідувачів для взаємодії з цими об'єктами.

Шаблон Memento

Проблема. Необхідно зафіксувати стан об'єкту для реалізації, наприклад, механізму відкату.

Рішення. Зафіксувати і винести (не порушуючи інкапсуляції) за межі об'єкта його внутрішній стан так, щоб згодом можна було його відновити в об'єкті. "Memento" зберігає внутрішній стан об'єкта "Originator" і забороняє доступ до себе всім іншим об'єктам окрім "Originator", який має доступ до всіх даних для відновлення в колишньому стані. "Caretaker" може лише передавати "Memento" іншим об'єктам.



Мал. 2. Структура шаблону Memento

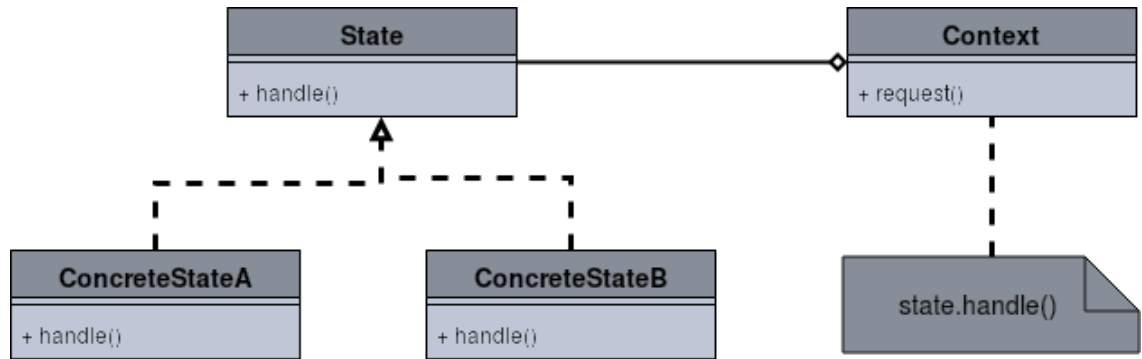
Учасники шаблону:

1. **Originator (Відновлювач)** – це клас, стан якого може бути збережений і відновлений. Він створює об'єкт "Memento" для збереження свого стану та відновлює свій стан із об'єкта "Memento".
2. **Memento (Запам'ятовувач)** – це об'єкт, який зберігає стан "Originator". Він містить стан "Originator" на певний момент часу.
3. **Caretaker (Опікун)** – це клас, який відповідає за збереження та відновлення "Memento". Він не повинен знати деталей структури "Memento" і "Originator", але він зберігає об'єкти "Memento" та використовує їх для відновлення стану "Originator".

Шаблон State

Проблема. Варіювати поведінку об'єкта в залежності від його внутрішнього стану.

Рішення. Клас "Context" делегує запити, які залежать від стану, поточному об'єкту "ConcreteState" (зберігає екземпляр підкласу "ConcreteState", яким визначається поточний стан), і визначає інтерфейс, що представляє інтерес для клієнтів. "ConcreteState" реалізує поведінку, асоційовану з якимось станом об'єкта "Context". "State" визначає інтерфейс для інкапсуляції поведінки, асоційованої з конкретним екземпляром "Context".



Мал. 3. Структура шаблону State

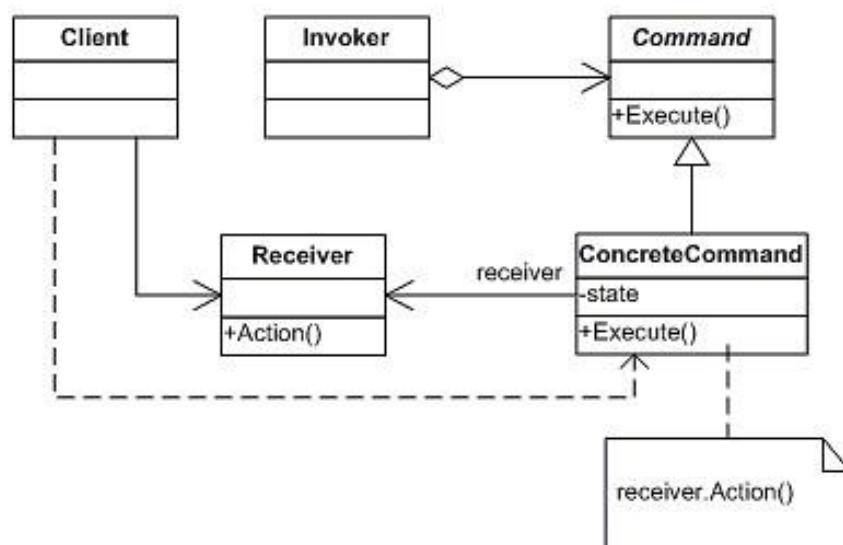
Учасники шаблону:

1. **Context (Контекст)** – це клас, який містить об'єкт, що має динамічно змінюваний стан. Він взаємодіє з об'єктом стану та викликає методи, пов'язані зі зміною стану.
2. **State (Стан)** – абстрактний клас або інтерфейс, який визначає методи, пов'язані з кожним станом. Кожен конкретний стан реалізує цей інтерфейс.
3. **ConcreteState (Конкретний стан)** – класи, які реалізують інтерфейс State та представляють конкретні стани об'єкта. Кожен конкретний стан визначає, як об'єкт повинен вести себе у цьому стані та які переходи між станами допустимі

Шаблон Command

Проблема. Необхідно надіслати об'єкту запит, не знаючи про те, виконання якої операції запитане і хто буде одержувачем.

Рішення. Інкапсулювати запит як об'єкт. "Client" створює об'єкт "ConcreteCommand", який викликає операції одержувача для виконання запиту, "Invoker" відправляє запит, виконуючи операцію "Command" Execute(). "Command" оголошує інтерфейс для виконання операції, "ConcreteCommand" визначає зв'язок між об'єктом "Receiver" і операцією Action(), і, крім того, реалізує операцію Execute() шляхом виклику відповідних операцій об'єкта "Receiver".



Мал. 4. Структура шаблону Command

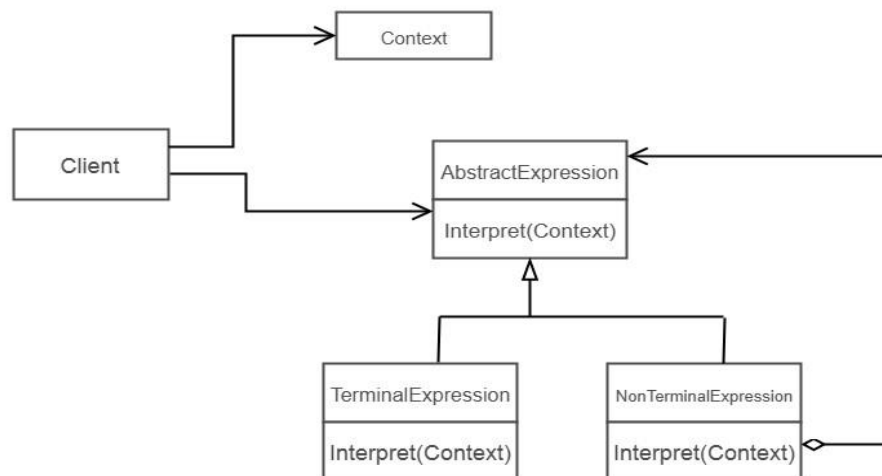
Учасники шаблону:

1. **Command (Команда)** – це інтерфейс або абстрактний клас, що описує специфікацію команди. Він містить методи виконання команди.
2. **ConcreteCommand (Конкретна команда)** – це клас, який реалізує інтерфейс Command та включає в себе конкретну логіку виконання команди. Він також може зберігати посилання на отримувача.
3. **Receiver (Отримувач)** – це клас, який має реальний функціонал та знає, як виконати команду.
4. **Invoker (Викликач)** – це клас, який відповідає за створення об'єктів команди та виклик їх виконання. Він також може управляти чергою команд.
5. **Client (Клієнт)** ініціює створення команд та налаштовує, які команди повинні бути виконані та коли.

Шаблон Interpreter

Проблема. Якщо якась задача виникає досить часто, є сенс подати її конкретні проявлення у вигляді речень простою мовою. Потім можна буде створити інтерпретатор, котрий вирішує задачу, аналізуючи речення цієї мови

Рішення. Створити інтерпретатор, який вирішує цю задачу. Завдання пошуку рядків за зразком може бути вирішено за допомогою створення інтерпретатора, що визначає граматику мови.



Мал. 5. Структура шаблону Interpreter

Учасники шаблону:

1. **Абстрактний синтаксичний аналізатор (Abstract Syntax Tree, AST)** – це структура даних, яка представляє абстрактний синтаксис мови. Вона складається з вузлів, які відповідають різним елементам мови.
2. **Термінальні та нетермінальні вирази.** Термінальні вирази представляють базові елементи мови, а нетермінальні вирази використовуються для поєднання інших виразів та визначення граматичних правил.
3. **Контекст** – об'єкт, що містить поточний стан вхідних даних і зберігає інформацію про стан обчислень.
4. **Інтерпретатори** – класи, які реалізують логіку обчислення значень виразів, відповідно до граматичних правил.

Завдання

1. Повторити шаблони поведінки для проектування ПЗ. Знати загальну характеристику шаблонів поведінки та призначення кожного з них.
2. Детально вивчити шаблони поведінки для проектування ПЗ – Visitor, Memento, State, Command, Interpreter. Для кожного з них:
 - вивчити Шаблон, його призначення, мотивацію, випадки коли його застосування є доцільним та результати такого застосування;
 - знати особливості реалізації Шаблону, споріднені шаблони, відомі випадки його застосування в програмних додатках;
 - вільно володіти структурою Шаблону, призначенням його класів та відносинами між ними;
 - вміти розпізнавати Шаблон в UML діаграмі класів та будувати сирцеві коди Java-класів, що реалізують шаблон.
3. В підготованому проєкті створити програмний пакет work5. В пакеті розробити інтерфейси і класи, що реалізують завдання 1 та 2 (згідно варіанту) з застосуванням одного чи декількох шаблонів (п.2). В класах, що розробляються, повністю реалізувати методи, пов'язані з функціонуванням Шаблону. Методи, що реалізують бізнес-логіку, закрити заглушками з виводом на консоль інформації про викликаний метод та його аргументи.

Приклад реалізації бізнес-методу:

```
void draw(int x, int y) {  
    System.out.println("Метод draw з параметрами x="+x+" y="+y);  
}
```

4. За допомогою автоматизованих засобів виконати повне документування розроблених класів (також методів і полів), при цьому документація має в достатній мірі висвітлювати роль певного класу в загальній структурі Шаблону та особливості конкретної реалізації.

Варіанти до завдання 1 (2 бали)

Номер варіанту завдання обчислюється як залишок від ділення номеру залікової книжки на 8.

ВАРІАНТ 0. Необхідно створити модель графічного інтерфейсу, що складається з таких типів елементів:

- **Button** – простий елемент (лист дерева), який не містить підлеглих елементів;
- **Panel** – контейнер (композит), який може містити інші елементи (Button або вкладені Panel).

Структура повинна підтримувати виконання довільних операцій (наприклад, підрахунок елементів певного типу, відображення структури тощо) без внесення змін у класи Button і Panel.

Вимоги:

- Створити абстрактний клас або інтерфейс **UIElement**, який описує спільну поведінку всіх елементів.
- Реалізувати класи:
 - Button – представляє кнопку, реалізує інтерфейс **UIElement**;
 - Panel – реалізує інтерфейс **UIElement** і може містити список підлеглих елементів (**List<UIElement>**).

- Розробити механізм розширення поведінки елементів інтерфейсу, який дозволяє виконувати над ними додаткові операції (наприклад, підрахунок, відображення тощо) без зміни коду класів **Button** та **Panel**. Для цього передбачити узгоджений набір методів, що забезпечують обробку кожного типу елемента.
- Створити окремий клас, який реалізує певну дію над елементами графічного інтерфейсу – наприклад, обчислення кількості кнопок або панелей. Цей клас повинен мати окремі методи для роботи з різними типами елементів.
- Кожен елемент інтерфейсу повинен мати спеціальний метод для прийому обробника, який дозволяє “віддати себе” для виконання певної операції. Самі елементи при цьому не знають, що саме робить обробник – вони лише передають йому управління.

У методі `main` продемонструвати:

- створення ієрархічної структури інтерфейсу (панелі з вкладеними елементами);
- застосування підрахунку кількості кнопок та панелей;
- можливість додати новий тип операцій (наприклад, `Display` для виводу структури) без зміни коду класів **Panel** і **Button**.

ВАРІАНТ 1. Необхідно створити модель структури комп’ютера, що складається з таких типів компонентів:

- **Processor** – окремий елемент, який представляє центральний процесор;
- **Memory** – окремий елемент, який представляє оперативну пам’ять;
- **VideoCard** – окремий елемент, який представляє відеокарту;
- **Computer** – комплексний елемент, який може містити інші компоненти (процесор, пам’ять, відеокарту, а також додаткові підсистеми).

Структура повинна підтримувати виконання довільних операцій (наприклад, визначення загальної потужності, виведення інформації тощо) без внесення змін у класи компонентів (**Processor**, **Memory**, **VideoCard**, **Computer**).

Вимоги:

- Створити абстрактний клас або інтерфейс **ComputerPart**, який описує спільну поведінку всіх компонентів комп’ютера.
- Реалізувати класи:
 - **Processor** – представляє процесор, реалізує інтерфейс **ComputerPart**;
 - **Memory** – представляє пам’ять, реалізує інтерфейс **ComputerPart**;
 - **VideoCard** – представляє відеокарту, реалізує інтерфейс **ComputerPart**;
 - **Computer** – реалізує інтерфейс **ComputerPart** і може містити список підлеглих компонентів (`List<ComputerPart>`).
- Розробити механізм розширення поведінки компонентів, який дозволяє виконувати над ними додаткові операції (наприклад, розрахунок потужності, виведення характеристик тощо) без зміни коду класів компонентів. Для цього передбачити узгоджений набір методів, які забезпечують обробку кожного типу компонента.
- Створити окремий клас, який реалізує певну дію над компонентами комп’ютера – наприклад, обчислення загальної потужності, що споживається комп’ютером. Цей клас повинен мати окремі методи для роботи з різними типами компонентів (**Processor**, **Memory**, **VideoCard**, **Computer**).
- Кожен компонент комп’ютера повинен мати спеціальний метод для прийому обробника, який дозволяє “передати себе” для виконання певної операції.

Самі елементи при цьому **не знають**, яку саме дію виконує обробник – вони лише делегують йому управління.

У методі main продемонструвати:

- створення ієрархічної структури комп'ютера (комп'ютер, який містить процесор, пам'ять, відеокарту тощо);
- застосування операції визначення загальної потужності, що споживається комп'ютером;
- можливість додати новий тип операцій (наприклад, **Display** для виведення структури або **Diagnostics** для перевірки стану компонентів) **без зміни коду класів Computer, Processor, Memory і VideoCard.**

ВАРІАНТ 2.

Необхідно створити **модель мережевої структури**, що складається з таких типів елементів:

- **Cable** – елемент, який представляє мережевий кабель;
- **Server** – елемент, який представляє сервер;
- **Workstation** – елемент, який представляє робочу станцію;
- **Network** – комплексний елемент, який може містити інші елементи мережі (сервери, кабелі, робочі станції, а також вкладені підмережі).

Структура повинна підтримувати виконання **додаткових операцій** (наприклад, розрахунок вартості, відображення конфігурації тощо) **без внесення змін у класи елементів** (Cable, Server, Workstation, Network).

Вимоги:

- **Створити абстрактний клас або інтерфейс NetworkElement**, який описує спільну поведінку всіх елементів мережі.
- **Реалізувати класи Cable, Server, Workstation, Network.**
- **Розробити механізм розширення поведінки елементів мережі**, який дозволяє виконувати над ними нові дії (наприклад, обчислення вартості, вивід структури, перевірку стану тощо) **без зміни коду класів елементів.** Для цього передбачити **узгоджений набір методів**, які забезпечують можливість обробки кожного типу елемента.
- **Створити окремий клас**, який реалізує конкретну операцію над елементами мережі, наприклад, **розрахунок кошторису мережевої структури.** Цей клас повинен мати окремі методи для обробки різних типів елементів (Cable, Server, Workstation, Network).
- **Кожен елемент мережі повинен мати спеціальний метод для прийому обробника**, який дозволяє "передати себе" для виконання певної операції. При цьому елемент **не знає**, яку саме дію виконує обробник, він лише передає йому управління.

У методі main продемонструвати:

- створення ієрархічної структури мережі (мережа, яка містить сервер, робочі станції, кабелі та підмережі);
- виконання операції розрахунку кошторису всієї мережевої структури;
- можливість додати нові типи операцій (наприклад, **Display** для відображення структури або **CheckConnection** для перевірки стану з'єднань) **без зміни коду класів Network, Server, Workstation, Cable.**

ВАРІАНТ 3. Необхідно створити модель університетської структури, що складається з таких типів елементів:

- **Student** – елемент, який представляє студента університету;
- **Teacher** – елемент, який представляє викладача;
- **Department** – елемент, який представляє кафедру;
- **University** – комплексний елемент, який може містити інші структурні елементи (кафедри, викладачів, студентів, а також вкладені підрозділи).

Структура повинна підтримувати виконання **додаткових операцій** (наприклад, підрахунок кількості студентів, визначення загальної заробітної плати викладачів, відображення структури університету тощо) **без внесення змін у класи елементів** (Student, Teacher, Department, University).

Вимоги:

- **Створити абстрактний клас або інтерфейс UniversityElement**, який описує спільну поведінку всіх елементів університету.
- **Реалізувати класи Student, Teacher, Department, University**. Клас University повинен мати можливість містити колекцію підлеглих елементів.
- **Розробити механізм розширення поведінки елементів університету**, який дозволяє виконувати над ними **нові дії** (наприклад, обчислення кількості студентів, розрахунок загальної заробітної плати, виведення структури тощо) **без зміни коду класів елементів**.

Для цього передбачити **узгоджений набір методів**, що забезпечують можливість обробки кожного типу елемента.

- Кожен елемент університету повинен мати **спеціальний метод для прийому обробника**, який дозволяє “передати себе” для виконання певної операції. При цьому елемент **не знає**, яку саме дію виконує обробник – він лише передає йому управління.

У методі main продемонструвати:

- створення **ієрархічної структури університету** (університет, який містить кафедри, викладачів і студентів);
- виконання операції **підрахунку кількості студентів** у всьому університеті;
- можливість **додавати нові типи операцій** (наприклад, DisplayStructure для відображення ієрархії або SalarySum для підрахунку загальної зарплати викладачів) **без зміни коду класів University, Department, Teacher, Student**.

ВАРІАНТ 4. Необхідно створити модель автомобіля, що складається з таких елементів:

- **Engine** – двигун автомобіля, має стан (пошкодження), базову вартість ремонту та час на ремонт;
- **Wheel** – колесо, має стан, вартість ремонту та час на заміну;
- **Body** – корпус автомобіля, має стан (подряпини, вм’ятини), вартість ремонту та час на ремонт;
- **Car** – комплексний елемент, який може містити інші елементи автомобіля (двигун, колеса, корпус тощо).

Структура повинна підтримувати **виконання додаткових операцій** (наприклад, підрахунок вартості, оцінка часу ремонту, формування звіту) **без внесення змін у класи елементів**.

Вимоги:

- *Всі елементи автомобіля повинні мати спільний інтерфейс, що дозволяє виконувати над ними додаткові операції.*
- *Кожен елемент повинен мати механізм для прийому обробника операцій, який делегує виконання дії зовнішньому об'єкту. Створити механізм, що дозволяє виконувати додаткові операції над елементами автомобіля, не змінюючи їхніх класів. Цей механізм повинен забезпечувати обробку кожного типу елемента окремо.*
- *Реалізувати об'єкт, який виконує операцію **ремонт автомобіля**, наприклад, *RepairVisitor*. Цей об'єкт повинен мати можливість:*
 - *обчислювати загальну вартість ремонту;*
 - *оцінювати загальний час ремонту;*
 - *формувати звіт про стан кожного елемента.*

- *Додаткові дії (наприклад, діагностика, фарбування, модифікація) повинні додаватися без зміни коду класів *Car*, *Engine*, *Wheel*, *Body*. Кожен новий тип операції реалізується окремо через механізм обробника.*

У методі *main* продемонструвати:

- *Створення автомобіля зі всіма компонентами (двигун, колеса, кузов).*
- *Виконання операцій через механізм обробника: розрахунок загальної вартості ремонту; оцінка загального часу ремонту; генерація звіту про стан кожного елемента.*
- *Можливість легко додавати нові операції, не змінюючи існуючі класи елементів.*

ВАРІАНТ 5. *Необхідно створити модель векторного редактора, який працює з графічними елементами:*

- ***Circle (коло)*** – елемент з атрибутами: колір, позиція, радіус;
- ***Triangle (трикутник)*** – елемент з атрибутами: колір, позиція, розмір;
- ***Rectangle (прямокутник)*** – елемент з атрибутами: колір, позиція, ширина та висота;
- ***GraphicEditor*** – комплексний елемент, який містить усі графічні об'єкти та дозволяє їх зберігати і відновлювати стан.

Вимоги:

- *Всі графічні елементи повинні реалізовувати **спільний інтерфейс**, який дозволяє виконувати операції редагування, отримувати та встановлювати стан.*
- *Кожен елемент повинен мати можливість:*
 - *створити знімок свого стану (колір, позиція, розмір);*
 - *відновити стан із збереженого знімка.*
- *Знімок зберігається у **відокремленому об'єкті**, який не порушує інкапсуляції елементів.*
- *Комплексний елемент редактора **GraphicEditor** управляє колекцією елементів та може:*
 - *зберігати стан усіх елементів одразу;*
 - *відновлювати стан усіх елементів до попереднього знімка;*
 - *підтримувати історію змін для можливості “Undo/Redo”.*
- *Механізм повинен працювати **універсально для всіх елементів**, не вимагаючи змін у їхніх класах.*

У методі *main* продемонструвати:

- *Створення кількох графічних елементів (коло, трикутник, прямокутник) з різними атрибутами (колір, позиція, розмір).*
- *Зміни атрибутів елементів та збереження стану після кожної зміни.*
- *Відновлення попередніх станів елементів (Undo).*

- Можливість додавати нові елементи або нові операції редагування, не змінюючи існуючий код елементів та механізму збереження стану.

ВАРІАНТ 6. Необхідно створити модель ігрового персонажа, який містить такі атрибути:

- **Position (позиція)** – координати персонажа в ігровому просторі;
- **Inventory (склад артефактів)** – набір предметів, які має персонаж;
- **HealthLevel (рівень здоров'я)** – кількість очок життя персонажа;
- **Character** – комплексний об'єкт, який об'єднує всі атрибути і надає механізм збереження та відновлення стану.

Вимоги:

- Персонаж повинен мати **методи для отримання та оновлення стану**, а також для взаємодії з механізмом збереження.
- Персонаж повинен мати можливість:
 - створювати **знімок свого поточного стану** (позиція, інвентар, рівень здоров'я);
 - відновлювати стан із збереженого знімка.
- Знімок стану зберігається у **відокремленому об'єкті**, який не порушує інкапсуляції персонажа.
- Система повинна підтримувати **історію станів персонажа**, що дозволяє реалізувати функціонал **Undo/Redo**.

У методі main продемонструвати:

- Створити персонажа з початковими атрибутами (позиція, інвентар, рівень здоров'я).
- Змінювати стан персонажа під час гри (збирати артефакти, змінювати позицію, отримувати шкоду).
- Зберігати стан після ключових змін.
- Відновлювати попередні стани персонажа за допомогою механізму Memento (Undo).
- Показати можливість додавання нових атрибутів або операцій без зміни існуючих класів персонажа.

ВАРІАНТ 7. Необхідно створити модель, яка працює з таблицею в базі даних:

- **Table (Таблиця)** – об'єкт, що містить набір рядків даних;
- **Row (Рядок)** – елемент таблиці, що містить значення полів;
- **TransactionManager** – клас, який управляє виконанням операцій над таблицею та організовує транзакції;
- **Знімок** – об'єкт, що зберігає попередній стан таблиці або її частини для можливості відкату.

Вимоги:

- Таблиця повинна підтримувати основні операції: вставка рядків (insert); оновлення рядків (update); видалення рядків (delete).
- Перед виконанням будь-якої операції над таблицею слід створювати **знімок поточного стану** (стан таблиці або її змінюваної частини). Знімок повинен зберігати **всі необхідні дані для відновлення таблиці** у разі відкату транзакції.
- **TransactionManager** керує виконанням серії операцій як однієї транзакції. Якщо операції виконуються успішно, стан знімка можна видалити або закомітити. У разі помилки або виклику rollback, TransactionManager відновлює таблицю з останнього збереженого стану.

У методі main продемонструвати:

- Створити таблицю та заповнити її декількома рядками.

- Виконати серію операцій (*insert, update, delete*) під управлінням *TransactionManager*.
- Зберігати стан таблиці перед кожною операцією.
- Виконати *rollback* транзакції і відновити таблицю до попереднього стану.

Варіанти до завдання 2 (2 бали)

Номер варіанту завдання обчислюється як залишок від ділення номеру залікової книжки на 7.

ВАРІАНТ 0. Необхідно створити модель **TCP-з'єднання**, яке може перебувати в одному з кількох станів:

- **LISTENING** – очікування вхідних підключень;
- **ESTABLISHED** – активне з'єднання;
- **CLOSED** – закрите з'єднання.

TCPConnection – клас, який представляє TCP-з'єднання і делегує виконання дій об'єкту, що описує поточний стан.

Вимоги

- Створити інтерфейс **ConnectionState**, який описує дії, які можуть виконуватися над TCP-з'єднанням (наприклад, *open()*, *close()*, *send(data)*, *receive()*).
- Кожен стан реалізує цей інтерфейс і визначає конкретну поведінку для цих операцій.
- Реалізувати окремі класи для кожного стану TCP-з'єднання: *ListeningState*, *EstablishedState*, *ClosedState*. Кожен клас визначає, як TCP-з'єднання поводить себе у своєму стані (наприклад, у стані **CLOSED** відправка даних заборонена, у **LISTENING** не можна отримати дані тощо).
- Клас **TCPConnection** містить посилання на поточний стан. Усі дії **TCPConnection** делегуються поточному стану, що дозволяє **змінювати поведінку динамічно**, просто змінюючи стан об'єкта. **TCPConnection** повинен мати можливість **перемикатися між станами** в залежності від подій (відкриття з'єднання, закриття, прийом/відправка даних). Не використовувати великі умовні оператори (*if-else* або *switch*) для визначення поведінки.

У методі *main* продемонструвати:

- Створити TCP-з'єднання у стані **LISTENING**.
- Виконати дії *open()*, *send(data)*, *receive()*, *close()* та показати, що поведінка змінюється в залежності від поточного стану.
- Перемикати стан **TCPConnection** на **ESTABLISHED** і **CLOSED** та демонструвати зміну дій без використання умовних операторів.

ВАРІАНТ 1. Необхідно створити модель **графічного редактора**, який включає такі елементи:

- **Tool (Інструмент)** – абстрактний опис дії, що виконується при взаємодії з робочим простором (наприклад, *Pen*, *Brush*, *Eraser*, *Fill*).
- **ConcreteTool (Конкретний інструмент)** – конкретні реалізації інструментів, які визначають, що відбувається при натисканні миші або переміщенні курсору.
- **Canvas (Робочий простір)** – клас, що представляє область для малювання та делегує поведінку натискання кнопки миші поточному інструменту.

Вимоги:

- Створити інтерфейс або абстрактний клас **ToolState**, який описує поведінку при взаємодії користувача з робочим простором (*onMouseClicked()*, *onMouseDown()* тощо).
- Реалізувати окремі класи для кожного інструменту (*Pen*, *Brush*, *Eraser*, *Fill*), які реалізують **ToolState**. Кожен клас визначає, що відбувається при взаємодії користувача

з Canvas: Pen малює лінію; Brush малює мазок із заданою товщиною; Eraser стирає контент; Fill заливає область кольором.

- Клас **Canvas** містить посилання на поточний інструмент (ToolState). Усі дії користувача на Canvas делегуються поточному інструменту. Зміна інструменту динамічно змінює поведінку Canvas без використання умовних операторів. Canvas повинен мати можливість **змінювати поточний інструмент під час роботи** (наприклад, при виборі користувачем нового інструменту з панелі). Поведінка Canvas повністю визначається поточним станом (інструментом).

У методі main продемонструвати:

- Створити Canvas та декілька інструментів (Pen, Brush, Eraser, Fill).
- Виконати натискання та перетягування миші в Canvas і показати, що поведінка змінюється в залежності від вибраного інструменту.
- Змінювати інструменти динамічно та демонструвати нову поведінку без використання умовних операторів.

ВАРІАНТ 2. Необхідно створити модель обробки замовлення в онлайн-магазині, яка включає такі елементи:

• **OrderState (Стан замовлення)** – абстрактний опис поведінки замовлення при спробі виконати дії: `pay()`, `ship()`, `deliver()`, `cancel()`.

• **ConcreteState (Конкретний стан)** – конкретні реалізації станів замовлення, які визначають, що відбувається при виконанні дії:

- `NewState` – нове замовлення; можна оплатити (`pay()`) або скасувати (`cancel()`).
- `PaidState` – оплачено; можна відправити (`ship()`) або скасувати (`cancel()`).
- `ShippedState` – відправлено; можна доставити (`deliver()`).
- `DeliveredState` – доставлено; всі дії недоступні.
- `CancelledState` – замовлення скасоване; всі дії недоступні.

• **Order (Контекст)** – клас, який містить поточний стан замовлення (`OrderState`). Всі дії замовлення (`pay()`, `ship()`, `deliver()`, `cancel()`) делегуються поточному стану. Зміна стану відбувається динамічно через методи стану без використання умовних операторів.

Вимоги:

- Створити інтерфейс або абстрактний клас `OrderState` з методами для всіх дій над замовленням.
- Реалізувати класи `NewState`, `PaidState`, `ShippedState`, `DeliveredState`, `CancelledState` із конкретною поведінкою для кожної дії.
- Клас `Order` делегує всі дії поточному стану і дозволяє динамічно змінювати стан замовлення.

У методі main продемонструвати:

- Створення замовлення у стані `NewState`.
- Виконання дії `pay()` → перехід у стан `PaidState`.
- Виконання дії `ship()` → перехід у стан `ShippedState`.
- Виконання дії `deliver()` → перехід у стан `DeliveredState`.
- Можливість скасування замовлення на ранніх етапах (`New` або `Paid`) через `cancel()`.

ВАРІАНТ 3. Необхідно створити модель для роботи з чергою HTTP-запитів:

- **HttpRequest (HTTP-запит)** – об'єкт, що містить інформацію про запит: URL, метод (`GET`, `POST` тощо), пріоритет, ідентифікатор.
- **RequestQueue (Черга запитів)** – структура, яка зберігає HTTP-запити та забезпечує їх обробку.

- **Command (Інтерфейс команди)** – описує операцію над чергою (виконати і, при потребі, відмінити).
- **ConcreteCommand (Конкретні команди)** – реалізація операцій над чергою, наприклад:
 - *RemoveRequestCommand* – виключення запиту з черги;
 - *ChangePriorityCommand* – зміна пріоритету запиту і переміщення його в черзі.
- **Invoker (Менеджер команд)** – клас, який приймає команди та виконує їх; може зберігати історію виконаних команд для можливості відкату.

Вимоги:

- Кожна операція над чергою (видалення запиту, зміна пріоритету) інкапсулюється у окремий об'єкт-команду.
- *RequestQueue* не знає, які саме операції над нею виконуються, усе керування відбувається через команди.
- Менеджер команд зберігає історію виконаних команд, щоб за потреби можна було **відмінити останню дію** (наприклад, повернути видалений запит або відновити попередній пріоритет).

У методі main продемонструвати:

- Створити чергу HTTP-запитів та додати кілька запитів з різними пріоритетами.
- Виконати команду видалення одного запиту та показати, що черга оновилася.
- Виконати команду зміни пріоритету одного або кількох запитів і продемонструвати, що їхня позиція в черзі змінилася.
- Показати повернення видаленого запиту або попереднього пріоритету.

ВАРІАНТ 4. Необхідно створити модель взаємодії користувача з пунктами меню та кнопками інструментальної панелі, яка включає такі елементи:

- **Action (Дія)** – абстрактний опис реакції на натискання кнопки або вибір пункту меню.
- **ConcreteAction (Конкретна дія)** – конкретні реалізації дій, які визначають, що відбувається при натисканні (наприклад, *SaveFileAction*, *OpenFileAction*, *CopyAction*, *PasteAction*).
- **Invoker (Ініціатор)** – клас, який відповідає за «натискання» кнопок чи вибір пунктів меню і делегує виконання дій відповідним об'єктам *Action*.

Вимоги:

- Створити інтерфейс або абстрактний клас **Action**, який описує метод *execute()*.
- Реалізувати окремі класи для кожної дії (*SaveFileAction*, *OpenFileAction*, *CopyAction*, *PasteAction*), які реалізують метод *execute()*. Кожен клас визначає, яка поведінка відбувається при взаємодії користувача.
- Клас **Invoker** містить посилання на поточну дію. Натискання кнопки чи вибір пункту меню викликає метод *execute()* поточної дії. Зміна дії динамічно змінює реакцію *Invoker* без використання умовних операторів.
- Реалізувати можливість формування **макро-реакцій** – послідовностей дій, що виконуються під час одного виклику (*MacroAction*, який містить список об'єктів *Action* і при виконанні викликає *execute()* для кожної з них).

У методі main продемонструвати:

- Створити кілька дій (*Save*, *Open*, *Copy*, *Paste*) та *Invoker*.
- Виконати натискання кнопок та показати, що *Invoker* делегує поведінку поточній дії.
- Динамічно змінювати дію *Invoker* та демонструвати нову поведінку без умовних операторів.

- Створити макро-реакцію з кількох дій та продемонструвати її виконання.

ВАРІАНТ 5. Необхідно створити модель для розбору та обчислення алгебраїчних виразів, яка включає такі елементи:

- **Expression (Вираз)** – абстрактний клас або інтерфейс, який визначає метод `interpret()`, що обчислює значення виразу.
- Класи, які представляють базові елементи виразу:
 - `NumberExpression` – число.
 - `VariableExpression` – змінна (за потреби можна передавати значення змінної через контекст).
- Класи, які представляють операції над виразами:
 - `AddExpression` – додавання двох виразів.
 - `SubtractExpression` – віднімання двох виразів.
 - `MultiplyExpression` – множення двох виразів.
 - `DivideExpression` – ділення двох виразів.
- **Context (Контекст)** – клас, який може містити значення змінних, якщо використовуються `VariableExpression`.

Вимоги:

- Створити інтерфейс або абстрактний клас `Expression` з методом `interpret(Context context)`.
- Реалізувати класи (`NumberExpression`, `VariableExpression`) для представлення чисел та змінних.
- Реалізувати класи (`AddExpression`, `SubtractExpression`, `MultiplyExpression`, `DivideExpression`), які обчислюють результат, комбінуючи підвирази.
- Створити клас `Context`, який зберігає значення змінних та передає їх у метод `interpret()`.

У методі `main` продемонструвати:

- Створення виразів типу $3 + 5 * 2$ або $(x - 4) / y$.
- Обчислення значення виразу через метод `interpret()`.
- Можливість комбінувати вирази у складніші структури без зміни класів інтерпретатора.

ВАРІАНТ 6. Необхідно створити систему для розбору та обчислення логічних виразів, що складаються з операцій **AND**, **OR**, **NOT** та змінних, які можуть приймати значення `true` або `false`.

Елементи системи:

- **Expression (Вираз)** – абстрактний клас або інтерфейс, що містить метод `interpret(Context context): boolean`, який повертає логічне значення виразу.
- Класи для представлення базових елементів:
 - `VariableExpression` – змінна, значення якої береться з контексту.
 - `ConstantExpression` – константа `true` або `false`.
- Класи для логічних операцій:
 - `AndExpression` – логічне «і» двох виразів.
 - `OrExpression` – логічне «або» двох виразів.
 - `NotExpression` – логічне «не» одного виразу.
- **Context (Контекст)** – клас, який зберігає значення змінних ($x = true$, $y = false$ тощо) і передає їх у метод `interpret()`.

Вимоги:

- Створити інтерфейс або абстрактний клас *Expression* із методом *interpret(Context context)*.
- Реалізувати класи для змінних та констант (*VariableExpression*, *ConstantExpression*).
- Реалізувати нетермінальні класи для логічних операцій (*AndExpression*, *OrExpression*, *NotExpression*).
- Клас *Context* зберігає значення змінних та передає їх при інтерпретації.

У методі *main* продемонструвати:

- Створення виразу типу $(x \text{ AND } y) \text{ OR } (\text{NOT } z)$.
- Обчислення значення виразу через *interpret(context)*.
- Можливість комбінувати вирази у складніші логічні конструкції без зміни класів інтерпретатора.

Питання для самостійної перевірки

1. Класифікація шаблонів проектування ПЗ.
2. Призначення шаблонів поведінки для проектування ПЗ.
3. Коротка характеристика кожного шаблону поведінки.
4. Призначення шаблону Visitor.
5. Структура шаблону Visitor та його учасники.
6. Призначення шаблону Memento.
7. Структура шаблону Memento та його учасники.
8. Призначення шаблону State.
9. Структура шаблону State та його учасники.
10. Призначення шаблону Command.
11. Структура шаблону Command та його учасники.
12. Призначення шаблону Interpreter.
13. Структура шаблону Interpreter та його учасники.

Протокол

Протокол має містити титульну сторінку (з номером залікової книжки), завдання, роздруковку діаграми класів, розроблений програмний код та генеровану документацію в форматі JavaDoc. Файли вихідного коду (*.java) повинні бути додані до протоколу.

Список рекомендованих інформаційних джерел

1. Design Patterns: elements of reusable object-oriented software / Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. Indianapolis: - Addison-Wesley, 1994. - 417 p. ISBN: 0201633612.
2. Mark Grand. Patterns in Java: A Catalog of Reusable Design Patterns Illustrated with UML, Volume 1, 2nd Edition. - Wiley Publishing, 2002. - 480 p. ISBN: 0471258393
3. Design Patterns. URL: https://en.wikipedia.org/wiki/Design_Patterns
4. Design Pattern – Overview. URL: https://www.tutorialspoint.com/design_pattern/design_pattern_overview.htm
5. Behavioral pattern - Wikipedia. URL: https://en.wikipedia.org/wiki/Behavioral_pattern#:~:text=In%20software%20engineering%2C%20behavioral%20design,flexibility%20in%20carrying%20out%20communication
6. Behavioral Design Patterns. URL: <https://refactoring.guru/design-patterns/behavioral-patterns>
7. Behavioral patterns. URL: https://sourcemaking.com/design_patterns/behavioral_patterns
8. Behavioral Design Patterns. URL: <https://www.javatpoint.com/behavioral-design-patterns>
9. Behavioral Design Patterns. URL: <https://programmingline.com/software-design-patterns/behavioral-design-patterns>
10. Behavioral Patterns. URL: <https://howtodojava.com/design-patterns/behavioral/>